ARCHITECTURAL COMPARISON: MONOLITH, LAYERED ARCHITECTURE (MVC) AND CLEAN ARCHITECTURE IN SCALABLE SYSTEMS

COMPARATIVO ARQUITETURAL: MONÓLITO, ARQUITETURA EM CAMADAS (MVC) E CLEAN ARCHITECTURE EM SISTEMAS ESCALÁVEIS

Abstract: This paper presents an ongoing comparative study among three widely used architectural patterns in software systems development: Monolith, Layered Architecture (MVC), and Clean Architecture. The research addresses the growing need for architectural decisions grounded in empirical evidence, since many technical decisions are still based on subjective preferences or market trends. To conduct the analysis, a tripartite prototype is proposed: the same task management application developed three times, each following a distinct architectural pattern, hosted on Amazon Web Services (AWS) cloud infrastructure, with performance metrics collected via Apache JMeter and monitored through Amazon CloudWatch and Grafana. The research is currently at the architectural specification and experimental planning stage, and the metrics collection phase has not yet been carried out. The expected contribution is a set of evidence-based guidelines for architectural pattern selection according to the project profile and organizational context.

Keywords: software architecture; Clean Architecture; MVC; monolith; scalability.

Resumo: Este trabalho apresenta um estudo comparativo em desenvolvimento entre três padrões arquiteturais amplamente utilizados no desenvolvimento de sistemas de software: Monólito, Arquitetura em Camadas (MVC) e Clean Architecture. A pesquisa aborda a necessidade de escolhas arquiteturais fundamentadas em evidências empíricas, já que muitas decisões técnicas ainda se baseiam em preferências subjetivas ou tendências de mercado. Para conduzir a análise, propõe-se um protótipo tripartido: a mesma aplicação de gerenciamento de tarefas desenvolvida três vezes, cada uma seguindo um padrão arquitetural distinto, hospedado na infraestrutura em nuvem da Amazon Web Services (AWS), com coleta de métricas de desempenho pelo Apache JMeter e monitoramento via Amazon CloudWatch e Grafana. A pesquisa encontra-se em estágio de especificação arquitetural e planejamento experimental, e a coleta de métricas ainda não foi executada. A contribuição esperada é a proposição de diretrizes baseadas em evidências para a seleção de padrões arquiteturais conforme o perfil e o contexto organizacional do projeto.

Palavras-chave: arquitetura de software; Clean Architecture; MVC; monólito; escalabilidade.

1 INTRODUÇÃO

A transformação digital ampliou a demanda sobre a arquitetura dos sistemas de software, que precisam sustentar grande volume de solicitações simultâneas e disponibilidade constante. Segundo Bass, Clements e Kazman (2012), a arquitetura de software é o conjunto de estruturas necessárias para raciocinar sobre o sistema, compreendendo seus elementos, as relações entre eles e as propriedades de ambos. A escolha do padrão arquitetural é, portanto, uma decisão estratégica que afeta a evolução, a manutenção e a escalabilidade do sistema durante todo o seu ciclo de vida (Garlan & Shaw, 1993).

Padrões como a Arquitetura em Camadas, difundida no modelo MVC (*Model-View-Controller*), e a Clean Architecture, proposta por Martin (2017), oferecem formas sistematizadas de estruturar o código, favorecendo a divisão de responsabilidades, a testabilidade e a independência de tecnologias externas. Estudos recentes indicam que a escolha arquitetural afeta métricas objetivas, como a latência de resposta (Blinowski et al., 2022; Tapia et al., 2020), o que reforça a necessidade de investigação empírica antes da decisão.

1.1 Problema de pesquisa

O problema central desta pesquisa é a dificuldade de escolher o padrão arquitetural que ofereça o melhor equilíbrio entre facilidade de implementação, manutenibilidade e escalabilidade em sistemas de médio porte. Essa escolha costuma se apoiar em preferências pessoais, modismos ou relatos anedóticos (Fowler, 2002), sem suporte quantitativo, o que gera decisões subótimas com impacto na qualidade, na evolução e nos custos operacionais. Blinowski et al. (2022) mostraram que, em máquina única, o monólito supera os microsserviços em latência, resultado contraintuitivo que reforça a necessidade de medição rigorosa. Poucos estudos, contudo, comparam os três modelos sob as mesmas condições operacionais.

1.2 Objetivo

Analisar quantitativa e qualitativamente o desempenho, a integração e a manutenibilidade de sistemas construídos segundo os três padrões, por meio de um protótipo funcional tripartido hospedado na AWS, com resultados exibidos em um painel de métricas em tempo real.

1.3 Justificativa

Padrões inadequados produzem dívida técnica, custos crescentes de manutenção e dificuldade de escalonamento. Newman (2015) observa que a separação difusa de responsabilidades transforma aplicações simples em estruturas de difícil evolução. A literatura disponível compara majoritariamente monólitos e microsserviços, sem considerar a Clean Architecture como alternativa estruturalmente coesa (Richards, 2015).

2 REVISÃO DE LITERATURA

A arquitetura de software estabelece a organização de um sistema por meio de decisões estruturais e princípios de projeto. Garlan e Shaw (1993) a definem como o estudo da organização global dos componentes, de suas interações e dos princípios que governam o projeto e a evolução do sistema. O modelo monolítico, que reúne interface, lógica de negócio e persistência em um único artefato implantável, é historicamente o mais utilizado: proporciona baixa latência em contextos de baixa complexidade, mas limita o escalonamento seletivo e amplia o acoplamento à medida que o sistema cresce (Dragoni et al., 2017).

A Arquitetura em Camadas, popularizada pelo MVC, reduz o acoplamento por meio da separação horizontal entre apresentação, lógica de negócio e persistência, melhorando a organização do código e facilitando a escalabilidade (Tu, 2023). A Clean Architecture (Martin, 2017) isola as regras de negócio no núcleo de camadas concêntricas e aplica o princípio da inversão de dependências, o que favorece a testabilidade e a independência tecnológica.

A comparação empírica entre padrões arquiteturais é uma linha de investigação estabelecida, porém concentrada no eixo monólito–microsserviços. O trabalho metodologicamente mais próximo é o de Blinowski et al. (2022), que compararam, de forma controlada, uma aplicação monolítica e sua versão em microsserviços, mantendo constantes a funcionalidade e o ambiente de execução. Em máquina única, a versão monolítica apresentou menor latência, atribuída à comunicação em processo, enquanto a distribuída foi superior no escalonamento horizontal sob carga crescente. Esse achado fundamenta a adoção das mesmas métricas (tempo de resposta, *throughput* e consumo de memória) e a hipótese de menor latência do monólito em baixa carga. Tapia et al. (2020) seguiram desenho semelhante — mesma funcionalidade sob arquiteturas distintas, medida em condições equivalentes, com foco em consumo de recursos de hardware e latência —, procedimento que este trabalho replica.

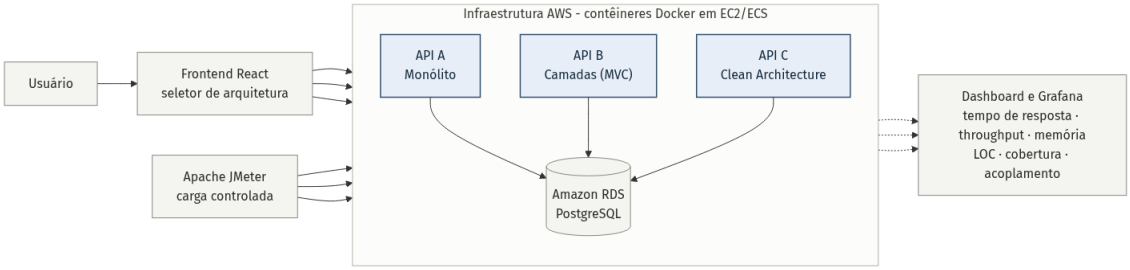
Revisões sistemáticas recentes reforçam essa leitura: Di Francesco et al. (2019) mapearam 103 estudos primários sobre arquitetura de microsserviços e evidenciaram a predominância desse eixo; Abgaz et al. (2023) analisaram 35 estudos sobre decomposição de monólitos e concluíram que o tema permanece em estágio inicial de maturidade; e Velepucha e Flores (2023) sistematizaram princípios e desafios de migração. Nenhum dos estudos identificados avalia empiricamente a Clean Architecture em comparação controlada com o monólito e com a Arquitetura em Camadas, sob identidade funcional, de linguagem e de infraestrutura — lacuna em que se insere este trabalho.

3 METODOLOGIA

A pesquisa adota a implementação de um protótipo tripartido: a mesma aplicação de gerenciamento de tarefas é desenvolvida três vezes, cada versão seguindo um padrão (Monólito, Arquitetura em Camadas/MVC e Clean Architecture). Assim, o padrão arquitetural é a variável independente, enquanto o domínio de negócio, a linguagem (TypeScript com Node.js), o banco de dados (PostgreSQL) e a infraestrutura (AWS) permanecem constantes. Cada versão contempla interface em React, API REST, regras de negócio para tarefas, autenticação por *JSON Web Tokens* (JWT) e persistência em PostgreSQL, diferindo apenas na organização do código-fonte e na distribuição de responsabilidades.

Cada instância é implantada em contêineres Docker, em instâncias Amazon EC2 de configuração idêntica, orquestrados pelo Amazon Elastic Container Service (ECS), com dados no Amazon RDS (PostgreSQL), segundo o AWS Well-Architected Framework (Amazon Web Services [AWS], 2023). Tempo de resposta, *throughput* e consumo de memória são coletados sob carga controlada gerada pelo Apache JMeter, com monitoramento no Amazon CloudWatch e painéis no Grafana, o que permite comparar diretamente as três versões (Figura 1). Indicadores de qualidade de código, como cobertura de testes, dependências entre módulos e linhas de código por camada, complementam a análise.

Figura 1 – Arquitetura da solução e instrumentação da coleta de métricas



Fonte: Elaborado pelos autores (2026).

4 RESULTADOS E DISCUSSÃO

A pesquisa está em desenvolvimento. Foram concluídas a revisão de literatura, a definição da arquitetura proposta e a especificação técnica do protótipo, incluindo a pilha tecnológica (TypeScript/Node.js, React, PostgreSQL) e os serviços AWS (EC2, ECS, RDS, S3, IAM e CloudWatch). A implementação das três versões e a coleta de métricas ainda não foram concluídas; portanto, não há dados quantitativos consolidados para as três arquiteturas.

Com base na literatura, formulou-se a hipótese que orientará a análise (Quadro 1): a Clean Architecture deve apresentar melhor testabilidade e menor acoplamento entre módulos; o Monólito, menor latência em baixa carga, replicando o resultado de Blinowski et al. (2022); e a Arquitetura em Camadas, o melhor equilíbrio entre simplicidade e organização para equipes menores. O Quadro 1 expressa expectativas derivadas da literatura, e não medições; elas serão confrontadas com os dados da fase experimental, sob condições controladas que eliminam variáveis externas.

Quadro 1 – Hipóteses de comportamento relativo entre os padrões avaliados

Critério	Monólito	Camadas (MVC)	Clean Architecture
Latência em baixa carga	Menor	Intermediária	Maior
Acoplamento entre módulos	Alto	Intermediário	Baixo
Testabilidade	Baixa	Intermediária	Alta
Complexidade de implementação	Baixa	Intermediária	Alta

Fonte: Elaborado pelos autores (2026), a partir da literatura revisada. Expectativas a verificar; não são resultados medidos.

5 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o desenho de um estudo comparativo, em desenvolvimento, entre Monólito, Arquitetura em Camadas (MVC) e Clean Architecture, com o objetivo de gerar diretrizes baseadas em evidências empíricas para a seleção do padrão mais adequado ao contexto de cada projeto. A contribuição consolidada até aqui é a delimitação da lacuna na literatura e o protocolo experimental, que isola o padrão arquitetural como única variável independente.

O protótipo tripartido permitirá coletar métricas objetivas de desempenho, qualidade de código e manutenibilidade sob condições controladas. As conclusões definitivas dependem da implementação das três versões e dos testes de carga, que constituem as próximas etapas. Como trabalhos futuros, propõem-se pipelines de integração e entrega contínua (CI/CD) para cada versão e a investigação da decomposição da Clean Architecture em microsserviços.

REFERÊNCIAS

Abgaz, Y., McCarren, A., Elger, P., Solan, D., Lapuz, N., Bivol, M., Jackson, G., Yilmaz, M., Buckley, J., & Clarke, P. (2023). Decomposition of monolith applications into microservices architectures: A systematic review. *IEEE Transactions on Software Engineering*, *49*(8), 4213–4242. https://doi.org/10.1109/TSE.2023.3287297

Amazon Web Services. (2023). *AWS Well-Architected Framework*. Recuperado em 20 de maio de 2026, de https://aws.amazon.com/architecture/well-architected/

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley.

Blinowski, G., Ojdowska, A., & Przybyłek, A. (2022). Monolithic vs. microservice architecture: A performance and scalability evaluation. *IEEE Access*, *10*, 20357–20374. https://doi.org/10.1109/ACCESS.2022.3152803

Di Francesco, P., Lago, P., & Malavolta, I. (2019). Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, *150*, 77–97. https://doi.org/10.1016/j.jss.2019.01.001

Dragoni, N., Giallorenzo, S., Lluch Lafuente, A., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), *Present and ulterior software engineering* (pp. 195–216). Springer.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Garlan, D., & Shaw, M. (1993). An introduction to software architecture. In V. Ambriola & G. Tortora (Eds.), *Advances in software engineering and knowledge engineering* (pp. 1–39). World Scientific Publishing.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Newman, S. (2015). *Building microservices: Designing fine-grained systems*. O'Reilly Media.

Richards, M. (2015). *Software architecture patterns*. O'Reilly Media.

Tapia, F., Mora, M. Á., Fuertes, W., Aules, H., Flores, E., & Toulkeridis, T. (2020). From monolithic systems to microservices: A comparative study of performance. *Applied Sciences*, *10*(17), Article 5797. https://doi.org/10.3390/app10175797

Tu, Z. (2023). Research on the application of layered architecture in computer software development. *Journal of Computing and Electronic Information Management*, *11*(3), 34–38. https://doi.org/10.54097/jceim.v11i3.08

Velepucha, V., & Flores, P. (2023). A survey on microservices architecture: Principles, patterns and migration challenges. *IEEE Access*, *11*, 88339–88358. https://doi.org/10.1109/ACCESS.2023.3305687